ChatGPT

Introduction

This document defines the remaining work required to complete your personal multi-agent research assistant. It consolidates all of the suggestions from our previous discussions into a single, actionable set of instructions. The goals are to remove the remaining confusion in the architecture, finish wiring up the embedding and classification stack, and provide a single search toolbox that the planner can call. Follow these steps carefully; do **not** skip any item.

---

1. Embedding Models and Classifiers

1.1 Embedding Models (TEI)

You will run two local embedding models on your 32 GB Ryzen 9 machine using the HuggingFace **Text-Embeddings-Inference** (TEI) service. These models provide dense vectors for retrieval:

Alias	HF Model	Dimensions	Purpose
emb-general	`thenlper/gte-large`	1024	General documents
emb-legal	`nlpaueb/legal-bert-base-uncased`	768	Legal documents only

You will **NOT** install other encoders (E5/BGE) or remote embeddings. The existing code embedding (`emb-code`) is untouched and remains mapped to your current provider. All documents will be embedded with **emb-general**. Legal documents receive a second embedding with **emb-legal**, and code receives a second pass with `emb-code` .

1.2 Classification Models (Ollama / Gemini)

Three classification models remain as-is. They are not encoders; they output discrete labels and confidence. Do **not** remove or rewire them now. They are:

- `emotion-english-distilroberta-base` (8-emotion general model) – returns one of anger, disgust, fear, joy, neutral, sadness, surprise, with confidence.
- `twitter-roberta-base-emotion` (TweetEval model) – returns one of anger, joy, optimism, sadness, fear, love, surprise.
- `twitter-roberta-base-sentiment-latest` – returns one of positive, neutral, negative.

These are already wrapped in Ollama with strict JSON prompts. Keep them unchanged. You will run them once per chunk during ingestion and store their labels in the database for future filtering. Do **not** embed with these models.

---

2. Docker Compose Services

Add TEI services to your `docker-compose.yml` file to host the embedding models locally. Append the following service definitions:

```
services:
  tei-gte-large:
    image: ghcr.io/huggingface/text-embeddings-inference:cpu-0.7
    environment:
      - MODEL_ID=thenlper/gte-large
    ports: ["6060:80"]
    healthcheck:
      test: ["CMD", "curl", "-sf", "http://localhost:80/health"]
      interval: 10s
      timeout: 3s
      retries: 10

  tei-legal:
    image: ghcr.io/huggingface/text-embeddings-inference:cpu-0.7
    environment:
      - MODEL_ID=nlpaueb/legal-bert-base-uncased
    ports: ["6061:80"]
    healthcheck:
      test: ["CMD", "curl", "-sf", "http://localhost:80/health"]
      interval: 10s
      timeout: 3s
      retries: 10
```

After adding these services, start them and verify they are healthy:

```
docker compose up -d tei-gte-large tei-legal
curl -sf http://localhost:6060/health && echo OK
curl -sf http://localhost:6061/health && echo OK
```

---

3. LiteLLM Configuration

Update `ops/litellm/config.yaml` so that the embedding aliases point to the TEI services. Remove any existing mappings for `emb-general` or `emb-legal`. At the end of the `model_list`, add:

```
- model_name: emb-general
  litellm_params:
    model: http://tei-gte-large:80
```

```
    provider: text-embeddings-inference

- model_name: emb-legal
  litellm_params:
    model: http://tei-legal:80
    provider: text-embeddings-inference
```

Do **not** modify the existing `emb-code` mapping.

After editing the file, restart the LiteLLM proxy:

```
docker compose restart litellm
```

---

4. Database Schema Changes

4.1 Embedding Spaces and Indices

Create new embedding spaces and add partial HNSW indices for the new spaces. Save the following SQL as `ops/scripts/sql/2025_embed_spaces_gte_legal.sql` and run it via psql:

```
-- Create spaces if they do not exist
INSERT INTO kb_embedding_space (name, provider, model, dims, distance)
SELECT 'emb-general','tei','thenlper/gte-large',1024,'cosine'
WHERE NOT EXISTS (SELECT 1 FROM kb_embedding_space WHERE name='emb-general');

INSERT INTO kb_embedding_space (name, provider, model, dims, distance)
SELECT 'emb-legal','tei','nlpaueb/legal-bert-base-uncased',768,'cosine'
WHERE NOT EXISTS (SELECT 1 FROM kb_embedding_space WHERE name='emb-legal');

-- Add indices to accelerate nearest-neighbour queries
DO $$
DECLARE sid_gen   int;
DECLARE sid_legal int;
BEGIN
  SELECT id INTO sid_gen   FROM kb_embedding_space WHERE name='emb-general';
  SELECT id INTO sid_legal FROM kb_embedding_space WHERE name='emb-legal';
  EXECUTE format('CREATE INDEX IF NOT EXISTS idx_hnsw_gen   ON
kb_chunk_embeddings USING hnsw (embedding vector_cosine_ops) WHERE
space_id=%s;', sid_gen);
  EXECUTE format('CREATE INDEX IF NOT EXISTS idx_hnsw_legal ON
kb_chunk_embeddings USING hnsw (embedding vector_cosine_ops) WHERE
```

```
space_id=%s;', sid_legal);
END $$;
```

Run the migration with:

```
psql "$DATABASE_URL" -f ops/scripts/sql/2025_embed_spaces_gte_legal.sql
```

4.2 Chunk Labels Table

Add a table to store the outputs of the classification models. Execute:

```
CREATE TABLE IF NOT EXISTS kb_chunk_labels (
  chunk_id UUID REFERENCES kb_chunks(id) ON DELETE CASCADE,
  model TEXT NOT NULL,
  label TEXT NOT NULL,
  score DOUBLE PRECISION NOT NULL,
  source TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (chunk_id, model, label)
);
CREATE INDEX IF NOT EXISTS idx_chunk_labels_label ON kb_chunk_labels(label);
CREATE INDEX IF NOT EXISTS idx_chunk_labels_model ON kb_chunk_labels(model);
```

4.3 Optional Features Table

If you plan to capture additional metrics (toxicity, readability, etc.), create a separate table:

```
CREATE TABLE IF NOT EXISTS kb_chunk_features (
  chunk_id UUID REFERENCES kb_chunks(id) ON DELETE CASCADE,
  feature_key TEXT NOT NULL,
  value JSONB NOT NULL,
  model TEXT NOT NULL,
  source TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (chunk_id, feature_key, model)
);
CREATE INDEX IF NOT EXISTS idx_chunk_features_gin ON kb_chunk_features USING
gin (value jsonb_path_ops);
```

---

5. Modelfile Updates (Strict JSON)

Hard-lock the JSON schemas in your classification wrappers to prevent hallucinated labels. Replace the `SYSTEM` blocks in each Modelfile as follows.

5.1 General Emotion

File: `ops/ollama/modelfiles/emotion-english-distilroberta-base/Modelfile`

```
SYSTEM """
You emulate j-hartmann/emotion-english-distilroberta-base for English emotion
classification.
Respond STRICT JSON ONLY—no text before/after—matching:
{"emotion": "anger|disgust|fear|joy|neutral|sadness|surprise", "confidence": 0.
0-1.0}
Rules:
- emotion MUST be exactly one of: anger, disgust, fear, joy, neutral, sadness,
surprise.
- Output exactly one JSON object.
"""
```

5.2 Twitter Emotion

File: `ops/ollama/modelfiles/twitter-roberta-base-emotion/Modelfile`

```
SYSTEM """
You emulate cardiffnlp/twitter-roberta-base-emotion (TweetEval). Respond STRICT
JSON ONLY—no text before/after—matching:
{"emotion": "anger|joy|optimism|sadness|fear|love|surprise", "confidence": 0.
0-1.0}
Rules:
- emotion MUST be exactly one of: anger, joy, optimism, sadness, fear, love,
surprise.
- Output exactly one JSON object.
"""
```

5.3 Twitter Sentiment

File: `ops/ollama/modelfiles/twitter-roberta-base-sentiment-latest/Modelfile`

```
SYSTEM """
You emulate cardiffnlp/twitter-roberta-base-sentiment-latest for tweet
sentiment analysis. Respond STRICT JSON ONLY—no text before/after—matching:
{"sentiment": "positive|neutral|negative", "confidence": 0.0-1.0}
```

```
Rules:
- sentiment MUST be exactly one of: positive, neutral, negative.
- Output exactly one JSON object.
"""
```

After editing all three Modelfiles, reload Ollama:

```
docker compose up -d ollama
docker compose up --abort-on-container-exit ollama-init
curl -s http://localhost:11434/api/tags
```

---

6. Ingestion Pipeline Changes

Update your ingestion scripts so that each chunk is embedded and classified properly.

1. **Embedding:** In `ops/scripts/embed_chunks.py`, set the spaces per chunk:
2. Always include `emb-general` for every chunk.
3. If the document domain is `legal`, append `emb-legal`.
4. If the document domain is `code`, append `emb-code`.

For each space in the list, call the LiteLLM `/v1/embeddings` endpoint, then insert a row into `kb_chunk_embeddings` with that `space_id` and the returned vector.

1. **Classify Once at Ingest:** Before embedding, run each classification model exactly once on the chunk's text. Call the wrapper via Ollama or LiteLLM with `format: json` and `stream: false` to get a JSON response. Insert a record into `kb_chunk_labels` with the chunk ID, model name, predicted label, score, and `source='ingest'`.

2. **Do NOT reclassify on user query.** The labels stored in `kb_chunk_labels` remain static and can be used later for filtering.

---

7. Search Toolbox API

Create a unified search API (or internal function) to handle retrieval. This replaces direct calls to DB keyword/vector/graph tools. The toolbox should perform:

1. **Query domain detection.** If the caller supplies `domain='auto'`, classify the user query as `general`, `legal`, or `code` using a lightweight classifier or simple heuristics. Otherwise, honour the provided domain.
2. **Query embeddings.** Always compute a vector with `emb-general`. If domain is `legal`, also compute with `emb-legal`. If domain is `code`, also compute with `emb-code`.
3. **Candidate generation:** Retrieve up to 50 rows from:

4. **BM25:** `tsv` full-text search.
5. **Vectors:** nearest neighbours in each active space (general + optional domain-specific).
6. **Combine with RRF.** UNION all candidates and compute a Reciprocal Rank Fusion score `1/(50 + rank)` per list. Sort by the sum across lists. Optionally filter by labels in `kb_chunk_labels` if a label filter is provided.
7. **Return a JSON response** containing `chunk_id`, final score, snippet, and the list of evidence spaces used.

The planner and workflow agent must call this toolbox instead of hitting the DB directly.

---

8. Planner and Workflow Changes

1. **Planner:** Remove raw DB, vector, or graph calls. For any information-seeking task, the planner should call the search toolbox with the user's question and appropriate flags (domain or label filters). The planner may still call classification tools for the user's query if needed to direct retrieval.

2. **Workflow Agent:** After retrieving ranked chunks via the toolbox, build an `EvidencePack` by selecting the top snippets (honouring the EvidencePack token cap). Pass this pack to the response agent to generate an answer.

3. **Graph Agent (Neo4j):** Implement two Cypher helpers for entity-driven tasks:

4. `graph_related_entities(name, k)` – returns k entities related to the given entity.
5. `graph_docs_for_entities(names[], k)` – returns documents mentioning all the entities in the list.

---

9. Permissions and Roles

Define database roles to enforce least privilege:

Role	Permissions
ingest_user	Insert into `kb_chunks`, `kb_chunk_embeddings`, `kb_chunk_labels`, call TEI and classifier APIs
search_user	Select from `kb_*` tables and call the search toolbox
agent_user	Insert into `agent.events` and read from `kb_*` tables
graph_user	Read/write in Neo4j via MCP; restricted network

Assign these roles to the appropriate containers and services.

---

10. Testing and Validation

1. **Unit Tests:** Write unit tests for the search toolbox to ensure correct ranking, RRF logic, and label filtering.
2. **Integration Tests:** Ingest a small corpus of general, legal, and code documents. Run embedding and classification. Query through the toolbox and verify that the results include the correct chunks and citations.
3. **Observability:** Log, for each query, which spaces were used, how many candidates came from each, the RRF score distribution, and whether any label filters were applied. Store these logs in `agent.events` or your preferred monitoring system.

---

11. Conclusion

By following these instructions, you will transform the current scaffolding into a coherent architecture. Two local TEI encoders (GTE-large and Legal-BERT) provide high-quality embeddings. Classification models produce labels once per chunk at ingestion and save them for later filtering. A unified search toolbox handles retrieval, combining BM25 with multiple embedding spaces via Reciprocal Rank Fusion. The planner interacts only with this toolbox and high-level graph utilities. Proper roles, indices, and logging make the system reliable, maintainable, and extensible.
